Second cerveau : méthode et architecture

De la capture à l'action, pour une base de connaissances personnelle durable

Auteur : Quentin Bourgois – Version 0.1 (brouillon) – Document d'explication et de réflexion. Il ne présente aucune mesure de productivité.

> **Convention de lecture.** Les affirmations suivies de [n] sont appuyées par une source listée en fin de document. Les passages marqués **(analyse de l'auteur)** sont des positions argumentées, non des faits sourcés. Les sources 1 à 3 sont des **résumés secondaires** de méthodes de praticiens (un blog personnel et deux pages d'éditeurs d'outils de notes ou de tâches, donc à finalité commerciale) : elles décrivent des méthodes, elles ne prouvent pas leur efficacité.

---

1. Contexte et question

Un « second cerveau » est un système externe où l'on dépose ce que l'on apprend, décide et planifie, pour le retrouver et l'utiliser plus tard. **(analyse de l'auteur)** Il devient plus intéressant avec les agents IA, qui peuvent lire, rédiger et mettre à jour ces notes.

La question de ce document : **quelle structure rend une base de connaissances personnelle durable, utilisable par un humain comme par un agent, sans la transformer en cimetière de notes ?**

2. Deux familles de méthodes

2.1 Orientée action : CODE et PARA. Tiago Forte propose le cycle CODE : Capture, Organize, Distill, Express.[1] Pour la capture, le conseil est de ne pas tout conserver, mais ce qui « résonne » ; l'organisation vise l'actionnabilité, avec la structure PARA (Projects, Areas, Resources, Archives) ; la distillation consiste à résumer la note par passes successives ; l'expression consiste à utiliser les notes pour créer ou agir.[1] D'après un résumé, PARA classe les informations par utilité immédiate : projets actifs, domaines récurrents, ressources, archives.[1]

2.2 Orientée pensée : Zettelkasten. Cette méthode consiste à écrire une idée par note (note « atomique ») et à la relier à d'autres notes, pour former un réseau plutôt qu'une hiérarchie de dossiers.[2] On y distingue en général des notes de lecture et des notes permanentes, qui contiennent l'interprétation personnelle de l'idée.[3] Les sources divergent sur la note de lecture : l'une la décrit

comme un résumé de la source,[3] l'autre comme la trace de la rencontre avec la source plutôt qu'un résumé.[2] Un guide d'éditeur recommande d'expliquer en une phrase pourquoi un lien existe.[2]

(analyse de l'auteur) Ces deux familles répondent à des besoins différents : PARA est un système de **gestion du travail en cours**, Zettelkasten un système de **construction de connaissance dans la durée**. Un second cerveau personnel gagne à utiliser PARA pour ce qui est actif et le principe des notes atomiques reliées pour ce qui doit survivre aux projets.

3. Le choix du format : fichiers avant application

Le principe « file over app » de Steph Ango (cofondateur d'Obsidian) soutient que les fichiers que l'on crée comptent plus que les outils qui les produisent : les applications sont éphémères, alors que les fichiers « ont une chance de durer », à condition d'être sous son contrôle et dans des formats faciles à relire.[4] Il écrit que ce sont ses fichiers texte brut, plus que l'application, qui sont conçus pour durer.[4] Un wiki communautaire ajoute que des notes Markdown se lisent avec n'importe quel éditeur, se traitent par script, se versionnent avec Git et se prêtent bien aux outils d'IA ; ces ajouts viennent de ce wiki et non de l'auteur du principe.[5]

(analyse de l'auteur) Pour un système destiné à durer des années et à être lu par des agents, c'est le critère le plus important : le contenu doit être lisible sans l'application qui l'a produit.

4. Ajouter un agent : la mémoire comme architecture

Un agent IA a une fenêtre de contexte limitée. MemGPT propose de la gérer comme une mémoire hiérarchique inspirée des systèmes d'exploitation : un contexte principal (le prompt) et un contexte externe, entre lesquels l'agent fait circuler l'information par appels de fonctions.[6] Une autre architecture, celle des *generative agents*, comporte trois composants : un flux de mémoire qui enregistre les expériences en langage naturel, une réflexion qui synthétise ces souvenirs en inférences de plus haut niveau, et la planification.[7] La récupération y combine pertinence, récence et importance.[7]

(analyse de l'auteur) Ces travaux concernent des agents de recherche, pas des notes personnelles. J'en retiens trois idées de conception applicables à un second cerveau :

1. séparer ce qui est **toujours chargé** (règles, préférences, tâches actives) de ce qui est **cherché à la demande** (archives, références) ;
2. distinguer les **faits bruts** des **synthèses**, comme les notes de lecture et les notes permanentes ;
3. ne pas retrouver au hasard : classer par pertinence, récence et importance.

5. Architecture proposée

(analyse de l'auteur) Quatre couches, toutes en fichiers Markdown :

1. **Capture** : une boîte d'entrée unique, où tout est déposé sans effort.
2. **Actif** : projets en cours et **une seule liste de tâches**. Chaque tâche a un propriétaire, un statut et une preuve d'achèvement.
3. **Connaissance** : notes atomiques reliées (concepts, décisions, procédures), écrites dans ses propres mots.
4. **Archive** : ce qui est terminé, avec la preuve d'achèvement, hors du chemin de recherche courant.

Règles transversales :

- **une source de vérité par type d'information** (une liste de tâches, pas trois) ;
- **un journal des actions**, pour qu'un agent ne modifie rien sans trace ;
- **ce qui est partiellement fait reste ouvert** : on n'archive que ce qui est vérifié ;
- **séparer faits vérifiés et hypothèses** dans les notes.

6. Rôle de l'agent et garde-fous

(analyse de l'auteur)

- L'agent **propose, résume, classe et rédige** ; l'humain valide les décisions et les suppressions.
- Les actions qui modifient l'état du système (archiver, supprimer, publier) sont journalisées et réversibles : l'historique Git est un filet de sécurité simple.
- Un agent ne doit pas pouvoir fusionner silencieusement deux versions d'une note.
- Les données sensibles (identifiants, données personnelles) n'ont pas leur place dans les notes lisibles par un agent, ou doivent être isolées.

7. Modes d'échec fréquents

(analyse de l'auteur)

- **Collecter sans traiter** : la boîte d'entrée grossit, rien n'est distillé. Conseil de [1] : ne capturer que ce qui résonne.
- **Sur-organiser** : passer plus de temps à classer qu'à agir. Conseil de [1] : organiser pour l'actionnabilité.

- **Notes composites** : plusieurs idées dans une note, qui ne peuvent être reliées que dans leur ensemble.[2]
- **Doublons de listes de tâches** : l'information se contredit.
- **Dépendance à un outil** : contenu enfermé dans un format propriétaire.[4]

8. Évaluation

(analyse de l'auteur) Je propose des signaux simples, sans prétendre à une mesure scientifique :

- le délai entre capture et traitement ;
- la part de tâches avec propriétaire, statut et preuve ;
- le nombre de notes jamais reliées ni relues ;
- le taux de retrouvailles : peut-on retrouver en moins d'une minute une décision prise il y a six mois ?

9. Limites de ce document

- Les méthodes CODE/PARA et Zettelkasten sont décrites ici d'après des résumés secondaires, pas d'après les ouvrages originaux, que je n'ai pas lus pour cette version.
- Aucune étude citée ne démontre que ces méthodes améliorent la productivité.
- MemGPT et les *generative agents* sont des travaux de recherche, dans des contextes différents d'un usage personnel.
- L'architecture de la section 5 est une proposition de l'auteur, issue d'une pratique personnelle, non validée par une étude.

10. Conclusion

Un second cerveau durable repose sur des **fichiers lisibles**, une **source de vérité unique pour l'action**, des **notes atomiques reliées** pour la connaissance et un **agent cantonné à proposer, avec traçabilité**. **(analyse de l'auteur)** La technologie compte moins que la discipline : traiter ce que l'on capture, et archiver seulement ce qui est réellement terminé.

Sources

1. Résumé de *Building a Second Brain* (CODE, PARA) – https://www.kuniga.me/books/2026/08/22/building-a-second-brain.html *(source secondaire)*

2. Guide Zettelkasten, Fabric (éditeur d'outil) – https://fabric.so/de/learn/zettelkasten *(source secondaire)*
3. Guide Zettelkasten, Todoist (éditeur d'outil) – https://www.todoist.com/productivity-methods/zettelkasten-method *(source secondaire)*
4. Steph Ango, *File over app* (source primaire) – https://stephango.com/file-over-app
5. PKM Wiki, *Local-First and Data Sovereignty* – https://pkm-wiki.knowii.net/local-first-and-data-sovereignty *(source secondaire)*
6. Packer et al., *MemGPT: Towards LLMs as Operating Systems*, arXiv:2310.08560 – https://arxiv.org/abs/2310.08560
7. Park et al., *Generative Agents: Interactive Simulacra of Human Behavior*, arXiv:2304.03442 – https://arxiv.org/html/2304.03442